

Warhammer40KApp Study Guide

Purpose: help you explain and defend the code, the React Native architecture, Redux, AsyncStorage, Firebase Auth, Firestore, forms, navigation, and modals.

Project: an Expo React Native app where users log in, create Warhammer 40K army compositions, add/remove units, track points, change theme, and manage profile settings.

Table of Contents

1. Expo, Assets, Fonts, and Splash Screen
2. TypeScript Types and Why They Matter
3. Reusable Components: Cards, Buttons, Header, Modal
4. Styling and Theme System
5. Theme Context and AsyncStorage
6. Authentication Context and Firebase Auth
7. Navigation: Custom Header, Stack, and Drawer Navigation
8. Forms, Formik, and Yup Validation
9. Forms Deep Dive
10. Firebase Services and Firestore Data Flow
11. Redux Store, Redux Toolkit, and Persisted State
12. Custom Hooks
13. App Structure and Startup Flow
14. Main Screens and User Flows
15. End-to-End Data Flow
16. TypeScript Defense Details
17. Security, Limitations, and Honest Defense
18. Defense Questions and Short Answers

1. Expo, Assets, Fonts, and Splash Screen

What Expo is

Expo is a platform and toolchain for React Native. It simplifies running, building, and configuring a React Native app. This project uses Expo scripts such as:

```
npm start
npm run android
npm run ios
npm run web
```

These scripts come from `package.json` and run Expo commands.

app.json

`app.json` contains Expo app configuration:

```
{
  "expo": {
    "name": "Warhammer40KApp",
    "slug": "Warhammer40KApp",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "plugins": [
      "expo-font"
    ]
  }
}
```

This file configures app name, icon, orientation, splash assets, platform settings, and plugins.

expo-font plugin

The app uses `expo-font` so it can load a custom font:

```
"plugins": [
  "expo-font"
]
```

The actual font loading happens in `useLoadFonts` :

```
export function useLoadFonts() {
  return useFonts({
    MedievalSharp: require("../assets/fonts/MedievalSharp-Regular.ttf"),
  });
}
```

The key `MedievalSharp` is the font family name the app can use in styles.

Why AppContent waits for fonts

```
const [fontsLoaded] = useLoadFonts();

if (loading || showSplashScreen || !fontsLoaded) {
  return <SplashScreen />;
}
```

The app waits until fonts are loaded before showing normal screens. This prevents text from appearing in a fallback font and then suddenly changing.

Splash screen behavior

The app has its own `SplashScreen` component and a short delay in `AppContent` :

```
const [showSplashScreen, setShowSplashScreen] = useState(true);

useEffect(() => {
  const timer = setTimeout(() => {
    setShowSplashScreen(false);
  }, 2000);

  return () => clearTimeout(timer);
}, []);
```

This keeps the splash screen visible briefly while startup checks finish. The cleanup clears the timer if the component unmounts early.

Local image assets

The app uses local images with `require` :

```
<ImageBackground
  source={require("../assets/images/screen_background.png")}
  style={styles.background}
/>
```

React Native bundles these local assets with the app.

armyImages helper

Firestore stores an image key, not the image file itself. The app maps image keys to local images through `armyImages` .

```
source={armyImages[armyComposition.army.imageKey]}
```

This is useful because Firestore stores simple serializable data, while the actual images stay bundled locally in the app.

Expo assets versus Firestore data

- Local assets are bundled with the app, such as icons, backgrounds, faction images, and fonts.
- Firestore stores dynamic data, such as armies and user-created army compositions.

Defense answer: Expo manages app configuration and development tooling. Local assets are bundled with the app, custom fonts are loaded with `expo-font`, and the app waits for fonts/auth/startup state before rendering the main UI.

2. TypeScript Types and Why They Matter

Army

```
export type Army = {  
  id: string;  
  name: string;  
  armyRule: string;  
  imageKey: ArmyImageKey;  
  units: Unit[];  
};
```

An army has an id, name, rule, image key, and list of units. The image key links Firestore data to local image assets.

Unit

```
export type Unit = {  
  id: string;  
  name: string;  
  movement: number;  
  toughness: number;  
  save: number;  
  wounds: number;  
  leadership: number;  
  objectControl: number;  
  ability: string;  
  points: number;  
};
```

This type describes the game stats for a unit.

ArmyComposition

```
export type ArmyComposition = {  
  id: string;  
  name: string;  
  army: Army;  
  units: SelectedUnit[];  
  totalPoints: number;  
};
```

An army composition is the user's saved list. It contains the selected army, selected units, and total points.

Why TypeScript helps

- It catches wrong property names before runtime.
- It documents the shape of the data.
- It makes navigation parameters safer.
- It improves autocomplete in the editor.

Defense answer: TypeScript gives the project stronger safety. It helps prevent common bugs, especially when passing data between screens, Redux, Firebase, and components.

3. Reusable Components: Cards, Buttons, Header, Modal

Component props

Props are values passed from a parent component to a child component. They make components reusable.

ArmyCompositionCard

```
type ArmyCompositionCardProps = {  
  armyComposition: ArmyComposition;  
  onPress: () => void;  
  onDelete: () => void;  
};
```

- `armyComposition` is the data to display.
- `onPress` is called when the user opens the card.
- `onDelete` is called when the user presses the trash icon.

UnitCard

```
type UnitCardProps = {  
  unit: Unit;  
  buttonText: string;  
  onPress: () => void;  
};
```

The same card can be used in two situations. In the modal, `buttonText` is `"+"`. In the selected unit list, it is `"-"`. This avoids duplicate components.

Modal

```
<Modal visible={visible} animationType="slide">
```

A React Native `Modal` renders content above the current screen. In this app, `SelectUnitModal` lets the user choose a unit without leaving the army composition screen.

```
type SelectUnitModalProps = {  
  visible: boolean;  
  units: Unit[];  
  totalPoints: number;  
  maxPoints: number;  
  onClose: () => void;
```

```
onAddUnit: (unit: Unit) => void;  
};
```

- `visible` controls whether the modal is shown.
- `units` is the list of valid units for the selected army.
- `totalPoints` and `maxPoints` show the point status.
- `onClose` closes the modal.
- `onAddUnit` sends the chosen unit back to the screen.

Defense answer: Reusable components reduce duplication. Props let each parent decide what data the component shows and what should happen when the user interacts with it.

4. Styling and Theme System

Style files

The project stores most styles in separate files under `src/styles`. This keeps screen and component files focused on structure and logic.

```
import { ThemeButtonStyles as styles } from "../../styles/themeButtonStyles";
```

The component imports its styles and uses them through the `style` prop.

StyleSheet.create

```
export const ThemeButtonStyles = StyleSheet.create({
  button: {
    paddingVertical: 12,
    paddingHorizontal: 16,
    borderRadius: 10,
    alignItems: "center",
    marginTop: 12,
  },
});
```

`StyleSheet.create` creates named style objects. React Native styling uses JavaScript objects instead of CSS classes.

Combining static and dynamic styles

```
<Pressable
  style={[styles.button, { backgroundColor: colors.button }]}
  onPress={onPress}
>
```

The style prop can receive an array. React Native combines the styles from left to right. Static layout comes from `styles.button`, while theme color comes from `colors.button`.

Theme colors

```
export const themeColors = {
  dark: {
    background: "#111",
    card: "#1E1E1E",
    text: "white",
    subText: "#B0B0B0",
    border: "#333",
    button: "#D9D9D9",
    buttonText: "black",
  },
};
```

```

    },
    light: {
      background: "#F2F2F2",
      card: "white",
      text: "black",
      subText: "#555",
      border: "#CCC",
      button: "#111",
      buttonText: "white",
    },
  },
};

```

The app uses one shared object for light and dark theme colors. Components read the current theme and choose the correct color set.

```

const { theme } = useTheme();
const colors = themeColors[theme];

```

This means changing the global theme value changes the color object used by the component.

ThemeButton

```

const ThemeButton = ({ theme, onPress }: ThemeButtonProps) => {
  const colors = themeColors[theme];

  return (
    <Pressable
      style={[styles.button, { backgroundColor: colors.button }]}
      onPress={onPress}
    >
      <Text style={[styles.buttonText, { color: colors.buttonText }]}>
        Switch to {theme === "dark" ? "Light" : "Dark"} Theme
      </Text>
    </Pressable>
  );
};

```

The button text is based on the current theme. If the current theme is dark, the button offers light mode. If the current theme is light, it offers dark mode.

ImageBackground

Many screens use `ImageBackground` to place content over a background image.

```

<ImageBackground
  source={require("../assets/images/screen_background.png")}
  style={styles.background}
>

```

```
...  
</ImageBackground>
```

This gives the app a consistent visual identity. In cards, overlays are used so text remains readable over faction images.

SafeAreaView

Screens use `SafeAreaView` so content avoids device notches, status bars, and bottom gesture areas.

```
<SafeAreaView  
  style={styles.container}  
  edges={["left", "right", "bottom"]}  
>
```

Defense answer: Styling is separated into style files for readability. Theme colors are centralized in `themeColors`, and components combine static styles with dynamic theme colors using style arrays.

5. Theme Context and AsyncStorage

Important file: `src/contexts/ThemeContext.tsx`

This file manages the light/dark theme globally.

```
type Theme = "light" | "dark";
```

This TypeScript union type means the theme can only be exactly `"light"` or `"dark"`. This prevents accidental values like `"blue"` or `"Dark"`.

```
type ThemeContextType = {  
  theme: Theme;  
  toggleTheme: () => void;  
};
```

The theme context exposes two things: the current theme value and a function to change it.

Loading saved theme

```
useEffect(() => {  
  const loadTheme = async () => {  
    const savedTheme = await AsyncStorage.getItem("theme");  
  
    if (savedTheme === "light" || savedTheme === "dark") {  
      setTheme(savedTheme);  
    }  
  };  
  
  loadTheme();  
}, []);
```

`AsyncStorage` is local key-value storage on the device. It is asynchronous, so the code uses `async` and `await`.

- `AsyncStorage.getItem("theme")` reads the saved value.
- The `if` check protects the app from invalid stored values.
- `setTheme(savedTheme)` updates the UI theme.
- The empty dependency array means this runs once when the provider mounts.

Toggling theme

```
const toggleTheme = async () => {  
  const newTheme = theme === "dark" ? "light" : "dark";  
  
  setTheme(newTheme);  
};
```

```
await AsyncStorage.setItem("theme", newTheme);  
};
```

The ternary operator means: if the current theme is dark, switch to light; otherwise switch to dark. The UI updates immediately with `setTheme`, then the new choice is saved locally.

Defense answer: I used AsyncStorage for the theme because it is a local user preference. It does not need to be stored in Firestore because it is not shared army data.

6. Authentication Context and Firebase Auth

Important file: `src/contexts/AuthContext.tsx`

This file creates a global authentication state. It lets every screen know whether a user is logged in.

```
type AuthContextType = {  
  currentUser: User | null;  
  loading: boolean;  
};
```

`currentUser` can be a Firebase `User` object or `null`. It is `null` when nobody is logged in. `loading` tells the app whether Firebase is still checking the saved login session.

```
export const AuthContext = createContext<AuthContextType>({  
  currentUser: null,  
  loading: true,  
});
```

`createContext` creates a shared data container. The object passed to it is the default value. The real values come from `AuthProvider`.

The provider

```
export const AuthProvider = ({ children }: AuthProviderProps) => {  
  const [currentUser, setCurrentUser] = useState<User | null>(null);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    const unsubscribe = onAuthStateChanged(auth, (user) => {  
      setCurrentUser(user);  
      setLoading(false);  
    });  
  
    return unsubscribe;  
  }, []);  
  
  return (  
    <AuthContext.Provider value={{ currentUser, loading }}>  
      {children}  
    </AuthContext.Provider>  
  );  
};
```

How Firebase Auth works here

- `onAuthStateChanged` starts listening to Firebase Auth.

- Firebase calls the callback when the user logs in, logs out, or when the app reloads and Firebase checks saved login data.
- The `user` parameter is either a Firebase user object or `null`.
- `setCurrentUser(user)` updates React state.
- `setLoading(false)` tells the app that the Firebase check is finished.

Why return unsubscribe?

`onAuthStateChanged` subscribes to auth changes and returns a function that stops the subscription. React calls a returned function from `useEffect` as cleanup.

```
return unsubscribe;
```

This is the same idea as:

```
return () => {  
  unsubscribe();  
};
```

Defense answer: I used Context for auth because login state is needed globally. The listener keeps the app synchronized with Firebase, and the cleanup prevents the listener from staying active after the provider is removed.

useAuth hook

```
export const useAuth = () => {  
  return useContext(AuthContext);  
};
```

This hook is a small wrapper around `useContext`. Instead of importing `useContext` and `AuthContext` everywhere, screens can simply call `useAuth()`.

7. Navigation: Custom Header, Stack, and Drawer Navigation

React Navigation concept

React Navigation controls which screen is visible. This project uses stack navigators for flows, a drawer navigator for large app sections, and a custom header component so the app has one consistent Warhammer-styled top bar.

- A **stack navigator** is like a pile of pages. You navigate forward, and you can go back.
- A **drawer navigator** is a side menu for switching between larger sections of the app.
- A **custom header** replaces the default React Navigation header with your own component.

NavigationContainer

In `App.tsx`, the app renders navigation only after auth, fonts, and the splash delay are finished:

```
<NavigationContainer>
  {currentUser ? <BattleForgeInformationDrawerStack /> : <AuthStack />}
</NavigationContainer>
```

`NavigationContainer` is the top-level navigation wrapper from React Navigation. It stores and manages navigation state. Inside it, the app chooses between the logged-out flow and the logged-in flow.

Defense answer: The navigation container wraps the navigators. The app conditionally renders the auth stack or main drawer depending on whether Firebase has a logged-in user.

AuthStack

```
export type AuthStackParamList = {
  Login: undefined;
  Register: undefined;
};
```

This type says the auth stack has two screens. `undefined` means those screens do not require route parameters.

```
<Stack.Navigator screenOptions={{ headerShown: false }}>
  <Stack.Screen name="Login" component={LoginScreen} />
  <Stack.Screen name="Register" component={RegisterScreen} />
</Stack.Navigator>
```

The header is hidden because login and register screens use their own layout.

Defense answer: The auth stack separates the logged-out screens from the main app. Login and Register do not need route params, so their param types are `undefined`.

BattleForgeStack

```
export type BattleForgeStackParamList = {
  Home: undefined;
  CreateArmyComposition: undefined;
  ArmyComposition: { armyComposition: ArmyComposition };
};
```

`ArmyComposition` needs data about the selected composition, so it receives an `armyComposition` route parameter.

```
<Stack.Screen
  name="Home"
  component={HomeScreen}
  options={{
    header: () => <Header title="Battle Forge" />,
  }}
/>
```

This screen uses the custom `Header`. Because `canGoBack` is not passed, it defaults to `false`, so the header shows the menu icon.

```
<Stack.Screen
  name="CreateArmyComposition"
  component={CreateArmyCompositionScreen}
  options={{
    header: () => <Header title="Create Army" canGoBack={true} />,
  }}
/>
```

This screen passes `canGoBack={true}`, so the custom header shows a back arrow instead of the drawer menu icon.

```
<Stack.Screen
  name="ArmyComposition"
  component={ArmyCompositionScreen}
  options={({ navigation }) => ({
    header: () => (
      <Header
        title="Army"
        canGoBack={true}
        onPress={() => navigation.navigate("Home")}
      />
    ),
  })
/>
```

```
    })}
  />
```

This screen uses a custom `onBackPressed` . Instead of simply going back in stack history, it navigates to `Home` . This avoids stacking duplicate detail screens.

InformationStack

```
export type InformationStackParamList = {
  InformationHome: undefined;
  GameRules: undefined;
  Profile: undefined;
  ArmyRules: undefined;
};
```

This stack groups the informational screens. The top-level `InformationHome` screen uses a menu header, while inner screens like Profile, Game Rules, and Army Rules use a back header.

```
<Stack.Screen
  name="Profile"
  component={ProfileScreen}
  options={{
    header: () => <Header title="Profile" canGoBack={true} />,
  }}
/>
```

This means Profile belongs to the Information stack and uses the shared custom header.

Drawer navigation

The drawer navigator is in `BattleForgeInformationDrawerStack.tsx` . It contains the two main logged-in sections of the app:

```
<Drawer.Screen
  name="BattleForgeStack"
  component={BattleForgeStack}
  options={{ title: "Battle Forge" }}
/>
```

```
<Drawer.Screen
  name="InformationStack"
  component={InformationStack}
  options={{ title: "Information" }}
/>
```

The drawer does not show individual screens like Home and Profile. Instead, it shows big sections. Each drawer item contains its own stack navigator.

The drawer also uses the theme:

```
const { theme } = useTheme();
const colors = themeColors[theme];
```

Those colors are used in `screenOptions` :

```
drawerStyle: {
  backgroundColor: colors.background,
},
drawerActiveBackgroundColor: colors.card,
drawerActiveTintColor: colors.text,
drawerInactiveTintColor: colors.subText,
```

This keeps the drawer visually consistent with the selected light or dark theme.

Why the drawer hides its own header

```
screenOptions={{
  headerShown: false,
}}
```

The drawer hides its default header because the inner stacks already provide headers through the custom `Header` component. This prevents two headers from appearing at the same time.

Custom Header component

The custom header lives in `src/screens/shared/Header.tsx` . It receives three props:

```
type HeaderProps = {
  title: string;
  canGoBack?: boolean;
  onBackPress?: () => void;
};
```

- `title` is the text shown in the header.
- `canGoBack` decides whether the left icon is a back arrow or a menu icon.
- `onBackPress` is an optional custom back action.

The header receives drawer navigation through `useNavigation` :

```
const navigation =
  useNavigation<
    DrawerNavigationProp<BattleForgeInformationDrawerStackParamList>
  >();
```

This lets the header call `navigation.openDrawer()` when the user presses the menu icon.

Header button logic

```
const handleMenuPress = () => {  
  if (canGoBack && onBackPress) {  
    onBackPress();  
  } else if (canGoBack && navigation.canGoBack()) {  
    navigation.goBack();  
  } else {  
    navigation.openDrawer();  
  }  
};
```

This function has three cases:

- If the screen can go back and a custom back function exists, run the custom function.
- If the screen can go back and navigation has a previous screen, use `navigation.goBack()`.
- Otherwise, open the drawer with `navigation.openDrawer()`.

That is why the same header component can work as both a drawer menu header and a back header.

Header icon logic

```
{canGoBack ? (  
  <Ionicons name="arrow-back" size={24} color="white" />  
) : (  
  <Ionicons name="menu" size={24} color="white" />  
)}
```

This is conditional rendering. If `canGoBack` is true, show the back arrow. Otherwise, show the menu icon.

Header layout

The header uses `SafeAreaView` with `edges={["top"]}` so it respects the top safe area on phones. It also uses `ImageBackground` so the header has the Warhammer-style background image.

```
<SafeAreaView style={styles.safeArea} edges={["top"]}>  
  <ImageBackground  
    source={require("../assets/images/header_background.png")}  
    style={styles.container}  
  >  
    ...  
  </ImageBackground>  
</SafeAreaView>
```

Defense answer: The app uses stacks for screen flows and a drawer for top-level sections. The drawer contains two stacks: Battle Forge and Information. Each stack uses the same custom Header component, which switches between opening the drawer and going back based on the `canGoBack` and `onBackPressed` props.

8. Forms, Formik, and Yup Validation

Why Formik?

Forms need to track input values, touched fields, validation errors, and submit handling. Formik manages that state so the component does not need many separate `useState` calls.

LoginForm example

```
<Formik
  initialValues={{ email: "", password: "" }}
  validationSchema={loginValidationSchema}
  onSubmit={values => {
    handleLogin(values.email, values.password);
  }}
/>
```

- `initialValues` defines the starting form values.
- `validationSchema` connects Yup validation rules.
- `onSubmit` runs after validation succeeds.

TextInput controlled by Formik

```
<TextInput
  value={props.values.email}
  onChangeText={props.handleChange("email")}
  onBlur={props.handleBlur("email")}
/>
```

The input value comes from Formik. When the user types, Formik updates the value. When the input loses focus, Formik marks the field as touched.

Yup validation

```
export const registerValidationSchema = Yup.object({
  email: Yup.string()
    .email("Enter a valid email")
    .required("Email is required"),

  password: Yup.string()
    .min(6, "Password must be at least 6 characters")
    .required("Password is required"),

  confirmPassword: Yup.string()
    .oneOf([Yup.ref("password")], "Passwords must match")
})
```

```
.required("Confirm password is required"),  
});
```

`Yup.ref("password")` means the confirm password must match the password field.

Small note: The message says `Passwods` . That is only a spelling issue in the validation message, not a logic problem.

Defense answer: Formik manages form state, and Yup keeps validation rules separate and reusable. This makes the form components cleaner.

9. Forms Deep Dive

Why forms use Formik

Formik manages form state, validation state, touched fields, and submit handling. Without Formik, each form would need multiple `useState` values and manual validation handling.

Formik structure

```
<Formik
  initialValues={{ email: "", password: "" }}
  validationSchema={loginValidationSchema}
  onSubmit={values => {
    handleLogin(values.email, values.password);
  }}
>
  {(props) => (
    ...
  )}
</Formik>
```

- `initialValues` defines the starting values.
- `validationSchema` connects Yup rules.
- `onSubmit` runs after the form is valid.
- `props` contains Formik helpers and form state.

Controlled TextInput

```
<TextInput
  value={props.values.email}
  onChangeText={props.handleChange("email")}
  onBlur={props.handleBlur("email")}
  autoCapitalize="none"
/>
```

This is a controlled input. The visible value comes from `props.values.email`. When the user types, `handleChange("email")` updates Formik's email value. When the field loses focus, `handleBlur("email")` marks the field as touched.

touched and errors

```
<Text style={styles.errorText}>
  {props.touched.email && props.errors.email
    ? props.errors.email
    : ""}
</Text>
```

`errors.email` contains the validation message. `touched.email` tells whether the user has interacted with the field. Showing an error only after a field is touched avoids displaying errors before the user has typed anything.

handleSubmit

```
<Pressable
  style={[styles.button, { backgroundColor: colors.button }]}
  onPress={() => props.handleSubmit()}
>
```

`handleSubmit` runs validation. If the data is valid, Formik calls `onSubmit`. If the data is invalid, Formik fills the `errors` object.

Yup validation

```
export const changePasswordValidationSchema = Yup.object({
  currentPassword: Yup.string().required("Current password is required"),

  newPassword: Yup.string()
    .min(6, "Password must be at least 6 characters")
    .required("New password is required"),

  confirmNewPassword: Yup.string()
    .oneOf([Yup.ref("newPassword")], "Passwords must match")
    .required("Confirm password is required"),
});
```

Yup schemas describe validation rules separately from the UI. `Yup.ref("newPassword")` compares one field to another, which is how confirm-password validation works.

Form ownership in this app

The form components own input rendering and validation. The screen components own the Firebase or app behavior after submit.

```
LoginForm
  -> validates email/password
  -> calls handleLogin(email, password)

LoginScreen
  -> receives handleLogin call
  -> calls Firebase auth service
  -> shows firebaseError if the request fails
```

This separation keeps forms reusable and keeps Firebase logic out of form components.

Keyboard handling

Form screens use `KeyboardAvoidingView` so the mobile keyboard does not cover inputs, and `TouchableWithoutFeedback` with `Keyboard.dismiss` so tapping outside the form hides the keyboard.

```
<TouchableWithoutFeedback onPress={Keyboard.dismiss}>
  <KeyboardAvoidingView
    behavior={Platform.OS === "ios" ? "padding" : "height"}
  >
    ...
  </KeyboardAvoidingView>
</TouchableWithoutFeedback>
```

Defense answer: Formik handles form state and submission, Yup handles validation rules, and the screens handle what happens after a valid submit. This keeps form UI separate from Firebase and navigation logic.

10. Firebase Services and Firestore Data Flow

Firestore config

```
const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const database = getFirestore(app);
```

`initializeApp` connects the app to your Firebase project. `auth` is used for login/register/logout. `database` is the Firestore database connection.

Auth service

```
export const login = (auth: Auth, email: string, password: string) => {
  return signInWithEmailAndPassword(auth, email, password);
};
```

The service function hides Firebase details from the screen. The screen calls `login`, and Firebase handles credential checking.

Firestore structure

```
users/{userId}/armyCompositions/{armyCompositionId}
```

This means each user has their own subcollection of army compositions. This is important because one user's armies should not appear for another user.

Live Firestore subscription

```
return onSnapshot(
  collection(database, "users", userId, "armyCompositions"),
  (querySnapshot) => {
    const armyCompositions = querySnapshot.docs.map((document) => {
      return document.data() as ArmyComposition;
    });

    onArmyCompositionsChanged(armyCompositions);
  },
);
```

`onSnapshot` listens in real time. When Firestore data changes, Firebase sends a new snapshot. The app converts Firestore documents into normal TypeScript objects and sends them back through the callback.

Saving data

```
return setDoc(  
  doc(database, "users", userId, "armyCompositions", armyComposition.id),  
  armyComposition,  
);
```

`doc` creates a reference to one Firestore document. `setDoc` writes data to that document.

Updating only units and total points

```
return setDoc(  
  doc(database, "users", userId, "armyCompositions", armyCompositionId),  
  { units, totalPoints },  
  { merge: true },  
);
```

`merge: true` means Firestore updates only the provided fields. It keeps other fields like `name` and `army`.

Defense answer: I put Firebase calls in service files to keep screens focused on UI. Services make the app easier to test, read, and change.

11. Redux Store, Redux Toolkit, and Persisted State

Important files

- `src/store/index.ts`
- `src/store/hooks.ts`
- `src/features/armyCompositions/armyCompositionSlice.ts`

What Redux does in this app

Redux stores the list of army compositions in one shared place. Screens can read from it without passing props through many layers.

Firestore is the remote database. Redux is the local app state. They work together.

Store setup

```
const rootReducer = combineReducers({  
  armyCompositions: armyCompositionsReducer,  
});
```

This means Redux state has a property called `armyCompositions`, controlled by `armyCompositionsReducer`.

```
const persistedReducer = persistReducer(persistConfig, rootReducer);
```

`redux-persist` wraps the reducer so Redux state can be saved to `AsyncStorage`.

Slice concept

```
export const armyCompositionSlice = createSlice({  
  name: "armyCompositions",  
  initialState,  
  reducers: {  
    armyCompositionAdded(state, action: { payload: ArmyComposition }) {  
      state.push(action.payload);  
    },  
  },  
});
```

A slice contains the state and the reducer functions for one feature. Redux Toolkit uses Immer internally, so code like `state.push` is allowed even though Redux state must stay immutable.

Important reducers

- `armyCompositionsLoaded` : replaces local Redux data with data loaded from Firestore.
- `armyCompositionsCleared` : empties the list when logging out.
- `armyCompositionAdded` : adds a new composition to Redux.
- `armyCompositionUnitsUpdated` : updates the units and total points of one composition.
- `armyCompositionDeleted` : removes a composition by id.

Typed Redux hooks

```
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();  
export const useAppSelector = useSelector.withTypes<RootState>();
```

These hooks make Redux usage type-safe. TypeScript knows what actions can be dispatched and what the state shape looks like.

Defense answer: Redux is useful here because army compositions are shared between multiple screens. Redux Toolkit reduces boilerplate and makes reducers easier to write safely.

12. Custom Hooks

What a custom hook is

A custom hook is a reusable function that uses React hooks or other app logic. Its name starts with `use`, such as `useAuth` or `useArmyCompositions`.

Custom hooks help keep screen files clean. Instead of putting Firebase, Redux, and state logic directly inside screens, the screen calls a hook and receives the values/functions it needs.

Screen

- > calls custom hook
- > hook handles state/services/Redux/Firebase
- > screen receives clean values and callbacks

Rules of hooks

- Hooks must be called at the top level of a component or another hook.
- Hooks should not be called inside loops, conditions, or nested functions.
- Custom hook names should start with `use` so React tooling recognizes them as hooks.

useAuth

```
export const useAuth = () => {  
  return useContext(AuthContext);  
};
```

`useAuth` is a small wrapper around `useContext(AuthContext)`. It lets screens write:

```
const { currentUser, loading } = useAuth();
```

instead of importing both `useContext` and `AuthContext` every time.

useTheme

```
export const useTheme = () => {  
  return useContext(ThemeContext);  
};
```

`useTheme` gives components access to the global theme and toggle function:

```
const { theme, toggleTheme } = useTheme();
```

useLoadFonts

```
export function useLoadFonts() {
  return useFonts({
    MedievalSharp: require("../assets/fonts/MedievalSharp-Regular.ttf"),
  });
}
```

This hook uses Expo's `useFonts` hook to load the custom font. `AppContent` waits for the returned `fontsLoaded` value before showing the normal UI.

useArmies

```
export function useArmies() {
  const [armies, setArmies] = useState<Army[]>([]);

  useEffect(() => {
    const loadArmies = async () => {
      const fetchedArmies = await getArmies(database);
      setArmies(fetchedArmies);
    };

    loadArmies();
  }, []);

  return { armies };
}
```

This hook loads the selectable armies from Firestore once. It uses `useState` to store the result and `useEffect` because loading data is a side effect.

useLoadArmyCompositions

```
export function useLoadArmyCompositions() {
  const dispatch = useAppDispatch();
  const { currentUser } = useAuth();

  useEffect(() => {
    if (!currentUser) {
      return;
    }

    const unsubscribe = subscribeToArmyCompositions(
      database,
      currentUser.uid,
      (armyCompositions) => {
        dispatch(armyCompositionsLoaded(armyCompositions));
      },
    );

    return unsubscribe;
  }, [currentUser]);
}
```

```
    }, [currentUser, dispatch]);
  }
```

This hook connects Firebase Auth, Firestore, and Redux. When a user is logged in, it subscribes to that user's Firestore army compositions. Whenever Firestore sends a new snapshot, the hook dispatches `armyCompositionsLoaded` into Redux.

useArmyCompositions

```
export function useArmyCompositions() {
  const dispatch = useAppDispatch();
  const { currentUser } = useAuth();
  const armyCompositions = useAppSelector(selectArmyCompositions);

  const deleteComposition = async (id: string) => {
    dispatch(armyCompositionDeleted(id));

    if (currentUser) {
      await deleteArmyComposition(database, currentUser.uid, id);
    }
  };

  return {
    armyCompositions,
    deleteComposition,
  };
}
```

This hook reads army compositions from Redux and exposes a delete function. The delete function first updates Redux, then deletes the document from Firestore. This is an optimistic UI update.

useCreateArmyComposition

This hook owns the logic for selecting an army and creating a new army composition.

```
const [selectedArmy, setSelectedArmy] = useState<Army | null>(null);
const [armySelectionError, setArmySelectionError] = useState("");
```

The hook tracks which army is selected and whether there is a selection error.

```
const newArmyComposition: ArmyComposition = {
  id: uuid.v4().toString(),
  name: compositionName,
  army: selectedArmy,
  units: [],
  totalPoints: 0,
};
```

When creation succeeds, the hook builds a new typed object with a unique id, empty units, and zero starting points.

```
dispatch(armyCompositionAdded(newArmyComposition));

await saveArmyComposition(database, currentUser.uid, newArmyComposition);

navigation.navigate("ArmyComposition", {
  armyComposition: newArmyComposition,
});
```

The hook updates Redux immediately, saves to Firestore, then navigates to the detail screen.

useArmyCompositionUnits

This hook manages adding/removing units and recalculating total points.

```
const [selectedUnits, setSelectedUnits] = useState<SelectedUnit[]>(
  armyComposition.units.map((unit) => ({
    ...unit,
    armyCompositionUnitId: uuid.v4().toString(),
  })),
);
```

The hook gives every selected unit copy its own `armyCompositionUnitId`. This matters because the same unit type can be added more than once.

```
const totalPoints = selectedUnits.reduce(
  (total, unit) => total + unit.points,
  0,
);
```

`reduce` calculates the total points from the selected units.

```
const saveUpdatedUnits = async (
  updatedUnits: SelectedUnit[],
  updatedTotalPoints: number,
) => {
  setSelectedUnits(updatedUnits);

  dispatch(
    armyCompositionUnitsUpdated({
      armyCompositionId: armyComposition.id,
      units: updatedUnits,
      totalPoints: updatedTotalPoints,
    }),
  );

  if (currentUser) {
    await updateArmyCompositionUnits(
```

```
        database,  
        currentUser.uid,  
        armyComposition.id,  
        updatedUnits,  
        updatedTotalPoints,  
    );  
}  
};
```

This helper updates three places: local screen state, Redux state, and Firestore.

Defense answer: Custom hooks keep the screens readable. Screens focus on UI, while hooks handle reusable logic such as reading context, loading Firestore data, dispatching Redux actions, managing unit state, and navigating after creation.

13. App Structure and Startup Flow

Important file: App.tsx

`App.tsx` is the root of the app. It decides which global providers exist and which navigation flow the user sees.

```
export default function App() {
  return (
    <Provider store={store}>
      <PersistGate loading={null} persistor={persistor}>
        <SafeAreaProvider>
          <AuthProvider>
            <ThemeProvider>
              <AppContent />
            </ThemeProvider>
          </AuthProvider>
        </SafeAreaProvider>
      </PersistGate>
    </Provider>
  );
}
```

How this works

- `Provider` comes from `react-redux`. It makes the Redux store available to the entire component tree.
- `PersistGate` waits until persisted Redux data has been loaded from local storage.
- `SafeAreaProvider` gives screens information about safe screen areas, such as notches and status bars.
- `AuthProvider` exposes `currentUser` and `loading` globally.
- `ThemeProvider` exposes `theme` and `toggleTheme` globally.
- `AppContent` can use those contexts because it is inside the providers.

Defense answer: Providers are placed high in the tree because many screens need access to the same global data. Authentication, theme, Redux state, and safe-area information are app-wide concerns, so they belong near the root.

AppContent

```
const AppContent = () => {
  const { currentUser, loading } = useAuth();
  const [fontsLoaded] = useLoadFonts();
  const [showSplashScreen, setShowSplashScreen] = useState(true);

  useLoadArmyCompositions();
}
```

```
useEffect(() => {
  const timer = setTimeout(() => {
    setShowSplashScreen(false);
  }, 2000);

  return () => clearTimeout(timer);
}, []);

if (loading || showSplashScreen || !fontsLoaded) {
  return <SplashScreen />;
}

return (
  <NavigationContainer>
    {currentUser ? <BattleForgeInformationDrawerStack /> : <AuthStack />}
  </NavigationContainer>
);
};
```

What each part means

- `useAuth()` reads the authentication context.
- `useLoadFonts()` loads the custom font before showing normal UI.
- `showSplashScreen` forces the splash screen to remain visible for about two seconds.
- `useLoadArmyCompositions()` starts syncing the user's Firestore army compositions into Redux when logged in.
- The `if` statement prevents the app from showing the wrong screen while auth or fonts are still loading.
- `NavigationContainer` is required by React Navigation. It manages navigation state.

Defense answer: The app waits for authentication, fonts, and the splash delay because showing the real app too early could cause flicker or briefly show the wrong navigation stack.

14. Main Screens and User Flows

LoginScreen and RegisterScreen

These screens show forms and call Firebase service functions.

```
const handleLogin = async (email: string, password: string) => {
  try {
    setFirebaseError("");
    await login(auth, email, password);
  } catch {
    setFirebaseError("Login failed");
  }
};
```

If login succeeds, Firebase Auth updates. Then `AuthContext` receives the new user through `onAuthStateChanged`. The app automatically switches from `AuthStack` to the main drawer.

HomeScreen

```
const { armyCompositions, deleteComposition } = useArmyCompositions();
```

The screen does not directly use Redux or Firestore. The custom hook gives it exactly what it needs: the list and a delete function.

CreateArmyCompositionScreen

This screen loads available armies from Firestore and lets the user choose one. The form collects the army composition name.

```
const {
  selectedArmy,
  armySelectionError,
  selectArmy,
  createArmyComposition,
} = useCreateArmyComposition(navigation);
```

The custom hook owns creation logic. The screen focuses on displaying army options and the form.

ArmyCompositionScreen

```
const { armyComposition } = route.params;
```

The selected army composition is passed through navigation parameters from the previous screen.

```
const { selectedUnits, totalPoints, isOverPointLimit, addUnit, removeUnit } =  
  useArmyCompositionUnits(armyComposition);
```

This hook manages unit selection, point calculation, Redux updates, and Firestore updates.

ProfileScreen

This screen uses auth, theme, Redux, and Firebase services.

```
const handleLogout = async () => {  
  dispatch(armyCompositionsCleared());  
  await logout(auth);  
};
```

It clears local Redux army data before signing out. This prevents another user from seeing the previous user's local army list.

Defense answer: The screens mostly compose UI and call hooks. Complex behavior is moved into hooks and services, so screens stay readable.

15. End-to-End Data Flow

Why data flow matters

For defense, it is useful to explain not only individual files, but also how the app behaves from one user action to the final UI update.

App startup flow

App starts

- > Redux Provider makes store available
- > PersistGate reloads persisted Redux data
- > AuthProvider starts Firebase auth listener
- > ThemeProvider loads saved theme from AsyncStorage
- > AppContent waits for auth, fonts, and splash delay
- > if currentUser exists, show main drawer
- > otherwise, show AuthStack

Login flow

User enters email/password

- > LoginForm submits values
- > LoginScreen calls login(auth, email, password)
- > Firebase Auth checks credentials
- > onAuthStateChanged receives user
- > AuthContext sets currentUser
- > AppContent re-renders
- > main drawer is shown

The important point is that `LoginScreen` does not manually navigate to the main app. Firebase auth state changes, `AuthContext` updates, and `AppContent` switches navigation.

Logout flow

User presses Sign out

- > ProfileScreen dispatches armyCompositionsCleared()
- > ProfileScreen calls logout(auth)
- > Firebase signs user out
- > onAuthStateChanged receives null
- > AuthContext sets currentUser to null
- > AppContent shows AuthStack

Redux is cleared before logout so the next user does not briefly see the previous user's local army data.

Create army composition flow

```
User opens Create Army
-> useArmies loads available factions from Firestore
-> user selects an army
-> user enters a composition name
-> Formik validates the name
-> useCreateArmyComposition builds newArmyComposition
-> dispatch(armyCompositionAdded(newArmyComposition))
-> saveArmyComposition writes to Firestore
-> navigation opens ArmyComposition screen
```

This flow combines Formik/Yup, hook state, Redux, Firestore, and navigation.

Add unit flow

```
User presses Add Unit
-> modalVisible becomes true
-> SelectUnitModal appears
-> user presses + on a UnitCard
-> addUnit creates a SelectedUnit with unique armyCompositionUnitId
-> total points are recalculated
-> local selectedUnits state updates
-> Redux armyCompositionUnitsUpdated is dispatched
-> Firestore document is updated with merge: true
-> UI re-renders with new unit and point total
```

Delete army composition flow

```
User presses trash icon
-> ArmyCompositionCard calls onDelete
-> HomeScreen calls deleteComposition(item.id)
-> Redux removes composition immediately
-> Firestore deleteDoc removes remote document
-> FlatList re-renders without that card
```

Theme toggle flow

```
User presses theme button
-> ThemeButton calls toggleTheme
```

- > ThemeContext calculates newTheme
- > setTheme updates UI immediately
- > AsyncStorage saves the new theme
- > components using useTheme re-render with new colors

Defense answer: The app's data flow is controlled by hooks and providers. User actions trigger callbacks, callbacks update state or dispatch Redux actions, Firebase/Firestore save persistent data, and React re-renders the UI from the latest state.

16. TypeScript Defense Details

Why TypeScript matters in this app

TypeScript is used to describe the shape of data and catch mistakes before runtime. This is especially useful because the app passes data through components, navigation, Redux, hooks, Firebase, and Firestore.

Union types

```
type Theme = "light" | "dark";
```

This means `theme` can only be `"light"` or `"dark"`. It prevents invalid values like `"blue"`, `"Dark"`, or an empty string.

Nullable types

```
const [currentUser, setCurrentUser] = useState<User | null>(null);
```

`User | null` means there may be a Firebase user, or there may be no user. This matches reality because before login there is no authenticated user.

Props types

```
type ThemeButtonProps = {  
  theme: "dark" | "light";  
  onPress: () => void;  
};
```

This says `ThemeButton` must receive a theme value and an `onPress` function. TypeScript warns you if the parent forgets one or passes the wrong type.

Optional props

```
type HeaderProps = {  
  title: string;  
  canGoBack?: boolean;  
  onBackPress?: () => void;  
};
```

The `?` means a prop is optional. `Header` can be used with only a title, or with back-button behavior.

Navigation param types

```
export type BattleForgeStackParamList = {
  Home: undefined;
  CreateArmyComposition: undefined;
  ArmyComposition: { armyComposition: ArmyComposition };
};
```

This protects navigation calls. TypeScript knows that `ArmyComposition` requires an `armyComposition` param, while `Home` does not.

RouteProp

```
type ArmyCompositionRouteProp = RouteProp<
  BattleForgeStackParamList,
  "ArmyComposition"
>;
```

This tells TypeScript what route params exist on the `ArmyComposition` screen. Then `route.params.armyComposition` is known to be an `ArmyComposition`.

StackScreenProps

```
type LoginScreenProps = StackScreenProps<AuthStackParamList, "Login">;
```

This gives the screen typed navigation props. It helps prevent mistakes in screen names and navigation parameters.

Type assertions with Firestore

```
return document.data() as ArmyComposition;
```

Firestore returns generic document data. The app tells TypeScript, "This document should have the shape of an `ArmyComposition`." This is useful, but it is also a trust point: TypeScript does not validate Firestore data at runtime.

Honest defense: TypeScript catches compile-time mistakes, but it does not prove that Firestore data is valid at runtime. For a larger production app, runtime validation could be added when reading Firestore documents.

Typed Redux hooks

```
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();
```

These hooks make Redux usage type-safe. TypeScript knows the dispatch type and the shape of the Redux state.

Defense answer: TypeScript documents the app's data shapes and catches many mistakes before the app runs. It is especially useful for props, navigation params, Redux state, Firebase users, and Firestore data models.

17. Security, Limitations, and Honest Defense

Why this chapter matters

In a project defense, it is strong to explain what the app does well and what could be improved. That shows you understand the system beyond simply making it work.

Firestore API keys

The Firebase config contains an API key. In Firebase frontend apps, this key is not treated like a private server secret. It identifies the Firebase project, but access should be controlled by Firebase Authentication and Firestore security rules.

Defense answer: Firebase frontend config is expected to be present in the client app. Real data protection should come from Firebase Auth and Firestore security rules, not from hiding the frontend API key.

Firestore security rules

The frontend stores user data under:

```
users/{userId}/armyCompositions/{armyCompositionId}
```

The security rule concept should be that users can only read and write their own documents:

```
allow read, write: if request.auth != null
  && request.auth.uid == userId;
```

The app builds the correct path with `currentUser.uid`, but Firestore rules are what enforce security on the backend.

Client-side validation limitation

Formik and Yup validate form input on the client. That improves user experience, but client-side validation can be bypassed. Important security rules should still exist on the backend or in Firestore rules.

Optimistic update limitation

The app often updates Redux before Firestore finishes. This makes the UI feel fast.

```
dispatch/armyCompositionDeleted(id));

if (currentUser) {
```

```
await deleteArmyComposition(database, currentUser.uid, id);
}
```

The limitation is that if Firestore fails, the UI has already changed. A stronger production version could catch the error and roll back the Redux state or show a retry message.

Runtime data validation limitation

The app uses TypeScript types for Firestore data:

```
return document.data() as ArmyComposition;
```

This tells TypeScript what shape the data should have, but it does not check the data at runtime. A larger app could validate Firestore data after reading it.

Error handling limitation

Some screens show general error messages like `Login failed` or `Registration failed`. This is simple and safe, but a more polished app could map Firebase error codes to clearer messages.

Data duplication tradeoff

An army composition stores the selected `army` object. This makes the detail screen easy to render because the selected composition already contains the army data. The tradeoff is that if the shared army data changes later, old compositions may still contain the old copy.

Good production improvements

- Add stronger Firestore security rules.
- Add rollback behavior for failed optimistic updates.
- Add runtime validation for Firestore documents.
- Add more specific Firebase error messages.
- Add loading and empty states for Firestore reads.
- Add tests for hooks, reducers, and validation schemas.

Defense answer: The current app meets the project requirements, but a production version should improve Firestore rules, error handling, rollback behavior, and runtime validation. Knowing these limitations shows understanding of real-world app quality.

18. Defense Questions and Short Answers

Why use Context for auth and theme?

Because both are global app state. Many screens need them, and passing them manually through props would be messy.

Why use Redux if you already use Context?

Context is good for small global values like auth and theme. Redux is better for feature state with multiple actions, such as adding, updating, loading, and deleting army compositions.

Why use AsyncStorage?

AsyncStorage saves small local data on the device. This app uses it for theme preference and Redux persistence.

Why use Firestore?

Firestore stores user army compositions remotely, so data is saved beyond one app session and can be synced from the backend.

Why use onSnapshot instead of getDocs for army compositions?

`getDocs` loads data once. `onSnapshot` keeps listening and updates the app when Firestore data changes.

Why use getDocs for armies?

The selectable armies are shared reference data. They only need to be loaded when the create screen needs them, so a one-time fetch is enough.

Why use Formik and Yup?

Formik manages form values and submit behavior. Yup defines validation rules clearly and separately.

Why give selected units a separate armyCompositionUnitId?

The same unit type can be added more than once. A separate selected-unit id lets the app remove one specific copy instead of all copies of the same unit.

Why clear Redux data on logout?

To prevent the next logged-in user from seeing local data from the previous user before Firestore finishes loading.

What is a hook?

A hook is a function that lets a component use React features such as state, context, and effects. Custom hooks in this project package reusable logic.

What is useEffect?

`useEffect` runs side effects, such as starting Firebase listeners, loading data, reading AsyncStorage, or setting timers. If it returns a function, React uses that function as cleanup.

What is state?

State is data that can change and cause the UI to re-render. Examples are `currentUser`, `loading`, `theme`, `selectedArmy`, and `modalVisible`.

What is the difference between props and state?

Props are passed into a component by its parent. State belongs to the component or hook and can change over time.

What is a reducer?

A reducer is a function that receives the current state and an action, then returns the next state. Redux Toolkit organizes reducers inside slices.

What is a modal?

A modal is a temporary UI layer displayed above the current screen. This app uses it to let the user choose a unit while staying inside the army composition flow.

Why use custom hooks?

Custom hooks keep screens focused on UI while reusable logic lives elsewhere. In this app, hooks handle context access, Firestore loading, Redux dispatching, army creation, unit management, and font loading.

What is the most important full data flow?

One example is: user creates army, Formik validates input, the custom hook builds the object, Redux updates immediately, Firestore saves it, and navigation opens the detail screen.

What is the role of Formik touched and errors?

`errors` stores validation messages, while `touched` tells whether the user interacted with the field. Showing errors only after a field is touched avoids noisy validation before the user types.

Why use TypeScript for navigation?

Typed navigation param lists prevent wrong screen names and missing route parameters. For example, TypeScript knows that `ArmyComposition` requires an `armyComposition` param.

How does the theme system work?

`ThemeContext` stores the current theme, `themeColors` maps that theme to colors, and components combine static styles with dynamic theme colors.

Are Firebase API keys secret?

Firebase frontend API keys are not treated like private server secrets. Real protection should come from Firebase Authentication and Firestore security rules.

What is an optimistic update?

An optimistic update changes the UI before the remote database finishes. It makes the app feel faster, but production apps should handle failures and possibly roll back the local state.

What does Expo do in this project?

Expo provides the React Native toolchain, app configuration, development scripts, asset handling, and font loading through `expo-font`.

Why wait for fonts before showing the app?

The app waits for fonts so screens do not render with a fallback font and then visually change after the custom font loads.